

ESCAPE FROM THE CURSED CASTLE

Project Report

Course: Introduction to Computer Science (ICS)
Course Code: CSL1010

Submitted by:

Pradeep Jangir (B24CH1029)
Vishesh (B24BB1045)
Kartik Meena (B24EE1032)
Sukhram (B24MT1037)

Submitted to:

Dr. Mayank Vatsa
Department of Computer Science

Academic Year 2024–25

1. Introduction

Escape from the Cursed Castle is a text-based role-playing game (RPG) built entirely in C for the Windows console. The game places the player inside a dangerous labyrinth, fighting through waves of monsters across multiple rounds before facing a powerful final boss. Every battle is driven by a type-effectiveness system, status effects, and a random-door mechanic that keeps each playthrough unpredictable.

This project was developed as part of the CSL1010 – Introduction to Computer Science course to demonstrate practical application of core C programming concepts in a complete, interactive program.

2. Objective of the Project

The main goal of this project was to design and implement a functional, feature-rich game in C that goes beyond basic programming exercises. Specific objectives include:

- Applying structs, arrays, and pointers in a real-world context
- Implementing a working turn-based combat system with meaningful game mechanics
- Building a progression system with XP, leveling, and difficulty scaling
- Demonstrating clean code structure across a ~1600-line program
- Making the console output visually appealing using Windows API color functions

3. Features of the Game

- **Multiple Difficulty Levels:** Three difficulty modes — Easy, Medium, and Hard — with distinct stat balancing
- **Rich Enemy Roster:** 24 unique enemies across 12 element types (Fire, Water, Ice, Rock, Flying, Electric, Earth, Plant, Dark, Psychic, Undead, Metal)
- **Combat System:** Turn-based combat with accuracy checks, critical hits, type advantages, and defense reduction
- **Status Effects:** 6 status effects: Burn, Poison, Paralysis, Freeze, Bleed, Confusion
- **Special Abilities:** 6 randomly triggered abilities: Critical Slash, Piercing Blow, Life Drain, Iron Wall, Counter Attack, Berserker Mode
- **Move Selection:** Player selects 5 moves from a pool of 33 typed attacks before the game begins
- **XP & Leveling:** XP system with visual progress bar; stats scale up on each level-up
- **Dynamic Enemy Scaling:** Enemies grow stronger each round; every 5th round spawns a mini-boss with a 25% stat bonus
- **Random Door Mechanic:** 2–5 mystery doors per round — may hide a fight, a reward, or a dead end
- **Shop System:** In-game shop where gold earned from battles can buy health and stamina potions
- **Final Boss Encounter:** Two final bosses (Abyssal Tyrant, Time Reaver) with 4 unique attacks each
- **Visual Presentation:** Animated typewriter-style text output with color coding using Windows console API

4. Concepts of C Programming Used

The following table summarises the key C programming concepts demonstrated in the project:

Concept	How It Is Used
Structures (struct)	Player, enemy, attack, ability, and status effect data are each stored in their own struct
Arrays of Structs	Enemy roster (24), attack pool (33), and ability list (6) are arrays of structs
Pointers & Pass-by-Reference	Player and enemy stats are modified through pointer parameters in battle functions
Preprocessor Macros (#define)	Color codes, XP bar length, type speed, and door limits defined as constants
String Functions (string.h)	strcmp / strcasecmp used for type matching, status checking, and name display
Random Number Generation	srand(time(0)) seeds rand() for enemy selection, accuracy, crits, and loot drops
Loops & Conditionals	Nested for/while loops drive battle rounds; if-else chains handle all game logic
File Includes & Windows API	windows.h for console color (SetConsoleTextAttribute) and Sleep() for animation
Floating-point Arithmetic	Type multipliers (0.5×, 1.0×, 2.0×) and level-up scaling (1.03f multiplier)
Input Validation	Invalid choices are caught and re-prompted in every menu and selection screen

5. Working / Flow of the Game

The game follows a clear round-based structure as described below:

- Player launches the game → views main menu → optionally reads the rulebook
- Selects difficulty (Easy / Medium / Hard) → stats and round count are set accordingly
- Chooses a name and picks 5 moves from the 33-move pool
- Each round: player stats UI is displayed → 2–5 mystery doors appear → player picks one
- Door outcome is random: enemy encounter, a reward (gold/potions/stat buff), or dead end
- In battles: player attacks first → accuracy and type effectiveness are checked → enemy retaliates → status effects tick each turn
- After each round: shop access, potion use, XP rewards, ability gain check
- Every 5th round: mini-boss encounter with boosted stats and doubled XP
- After all rounds are cleared: the Final Boss appears (Abyssal Tyrant or Time Reaver) with 700 HP and 4 powerful attacks
- Victory → congratulations screen; Player death at any point → game over screen

6. Important Functions / Modules

Function	Purpose	Concepts Used
battle()	Handles turn-by-turn combat loop between player and enemy	Structs, loops, conditionals
finalmonster()	Manages the final boss fight with 4 unique attacks	Struct arrays, randomness
select_moves()	Player picks 5 attacks from a pool of 33 moves	Arrays, input validation
gain_xp()	XP tracking, level-up logic, stat scaling with progress bar	Arithmetic, while loop
scale_enemy_stats()	Scales enemy HP/power based on current round number	Math functions, round logic
update_status_effect()	Applies Burn, Poison, Freeze, etc. per turn	Structs, string comparison
shop() / usePotion()	In-game shop and potion consumption system	User input, conditionals
select_door()	Random door system — combat, reward, or dead end	rand(), struct pointers
get_effectiveness()	Type chart multiplier lookup (12×12 type grid)	2D float array, strcasecmp

6.1 Special Ability Reference

Ability	Effect	Duration
Critical Slash	50% chance to deal double damage	1 round
Piercing Blow	Attacks bypass enemy defense	1 round
Life Drain	Heal for 50% of damage dealt	2 rounds
Iron Wall	Defense doubled	1 round
Counter Attack	Reflect 30% of damage taken back	1 round
Berserker Mode	ATK +50%, DEF -50%	1 round

6.2 Difficulty Mode Comparison

Mode	Player Stats	Enemy Stats	Rounds
Easy	HP: 600, Stamina: 250	-25% HP & power	10
Medium	HP: 500, Stamina: 200	Standard stats	15
Hard	HP: 400, Stamina: 150	+25% HP & power	20

7. Challenges Faced

- Managing status effects for both player and enemy through a shared struct without mixing up references was initially tricky — solved using explicit pointer checks inside `update_status_effect()`.
- Building the 12×12 type-effectiveness chart as a float array required careful indexing; type mismatches between uppercase attack types and lowercase enemy types were fixed using `strcasecmp()`.
- Balancing all three difficulty modes so that Hard felt genuinely punishing without being unfair took several rounds of stat tweaking.
- The door-selection logic required guaranteeing at least one valid progression path per round, which needed extra conditional logic to avoid trapping the player unfairly.
- Making console output visually appealing — with color, centered text, and typewriter animation — required learning and integrating the Windows API (`SetConsoleTextAttribute`, `Sleep`), which is not part of the standard C library.

8. Conclusion

Escape from the Cursed Castle is a complete, playable game that brings together nearly every concept taught in CSL1010. From basic I/O and conditionals all the way to structs, pointer manipulation, and Windows API integration, the project demonstrates how a large C program can be organized into modular functions with clearly separated responsibilities.

The end result is a game that feels genuinely engaging — with strategic move selection, risk-reward door choices, scaling difficulty, and satisfying progression — all built from scratch in C. This project significantly deepened my understanding of how data structures and algorithmic logic combine to create interactive software, and it was a valuable exercise in writing clean, maintainable code at scale.
